

Modules & Packages

Organise, Reuse and Publish Python Code

■ LEARNING OUTCOME

By the end of this module you will create and import modules, build Python packages, use the standard library effectively, manage dependencies with pip and virtual environments, and understand Python's import system.

■ Skills You Will Gain

- Create modules and import from them
- Build packages with `__init__.py`
- Use the most useful standard library modules
- Manage dependencies with pip and requirements.txt
- Create and use virtual environments
- Understand absolute vs relative imports

■ The Python standard library has modules for almost everything. Knowing what's available saves hours of writing code from scratch. The ecosystem of 400,000+ third-party packages on PyPI makes Python unstoppable.

SECTION 1

Modules and Imports

```
# IMPORTING MODULES import math # import entire module from math import sqrt, pi # import specific
names from math import * # import all (AVOID -- pollutes namespace) import numpy as np # import
with alias (conventional) from pathlib import Path # from package import class # Using imported
names print(math.sqrt(16)) # 4.0 print(sqrt(16)) # 4.0 (from math import sqrt) print(math.pi) #
3.14159... print(pi) # 3.14159... # CREATING YOUR OWN MODULE (utils.py) """ utils.py -- utility
functions for data processing """ def validate_email(email: str) -> bool: """Check if email has
basic valid format.""" return "@" in email and "." in email.split("@")[-1] def clamp(value,
minimum, maximum): """Return value clamped to [minimum, maximum] range.""" return max(minimum,
min(maximum, value)) if __name__ == "__main__": # Only runs when this file is executed directly #
NOT when imported as a module print(validate_email("alice@example.com")) # True print(clamp(150, 0,
100)) # 100 # IMPORTING YOUR MODULE # If utils.py is in the same directory: import utils
print(utils.validate_email("test@test.com")) from utils import validate_email, clamp
print(validate_email("bad-email")) # False
```

SECTION 2

Packages and Standard Library

```
# PACKAGE STRUCTURE # mypackage/ # __init__.py -- makes it a package, can export names # core.py --
core functionality # utils.py -- utility functions # models/ # __init__.py # user.py # product.py #
mypackage/__init__.py from .core import connect, disconnect from .utils import validate_email
__version__ = "1.0.0" __all__ = ["connect", "disconnect", "validate_email"] # public API # Using the
package import mypackage mypackage.connect() from mypackage import validate_email from
mypackage.models.user import User # deep import # ESSENTIAL STANDARD LIBRARY MODULES import os,
sys, time, datetime, random, json, re, math from pathlib import Path from collections import
Counter, defaultdict, deque from functools import partial, lru_cache, wraps from itertools import
chain, product, combinations, permutations from typing import List, Dict, Optional, Union, Tuple
from dataclasses import dataclass # os module print(os.getcwd()) # current directory
os.makedirs("dir", exist_ok=True) print(os.environ.get("HOME")) # environment variable
print(os.cpu_count()) # number of CPUs # sys module print(sys.argv) # command line arguments
print(sys.version) # Python version sys.exit(0) # exit program # datetime from datetime import
datetime, date, timedelta now = datetime.now() today = date.today() print(now.strftime("%Y-%m-%d
%H:%M:%S")) d = datetime.strptime("2024-01-15", "%Y-%m-%d") next_week = today + timedelta(days=7) #
random random.randint(1, 100) # random int in [1,100] random.random() # float in [0.0, 1.0)
random.choice([1,2,3,4,5]) # random item from sequence random.shuffle(my_list) # shuffle in place
random.sample(population, k=3) # k unique random items # itertools for comb in
combinations([1,2,3,4], 2): # (1,2),(1,3),(1,4)... print(comb) for perm in permutations("ABC", 2):
# all 2-char permutations print(perm)
```

SECTION 3

Virtual Environments and pip

```
# VIRTUAL ENVIRONMENTS -- isolate project dependencies # Create (run in terminal, NOT Python)
python -m venv venv # create virtualenv named "venv" # Activate source venv/bin/activate #
Linux/Mac venv\\Scripts\\activate # Windows # Deactivate deactivate # pip -- Python package
installer pip install requests # install package pip install pandas==2.1.0 # specific version pip
install "pandas>=2.0" # version constraint pip install -r requirements.txt # install from file pip
uninstall requests # remove package pip list # list installed pip show pandas # package info pip
freeze > requirements.txt # save current deps pip install --upgrade pip # upgrade pip itself #
requirements.txt example # requests==2.31.0 # pandas>=2.0.0, <3.0.0 # numpy>=1.24 #
python-dotenv==1.0.0 # POPULAR THIRD-PARTY PACKAGES # requests -- HTTP requests (simpler than
urllib) # pandas -- data analysis and manipulation # numpy -- numerical computing # matplotlib --
```

```

plotting and visualisation # sqlalchemy -- SQL ORM # pydantic -- data validation with type hints #
fastapi -- modern web API framework # pytest -- testing framework # black -- code formatter # mypy
-- static type checker import requests response =
requests.get("https://api.github.com/users/python") data = response.json() print(data["name"]) #
functools.lru_cache -- memoisation from functools import lru_cache @lru_cache(maxsize=None) def
fibonacci(n): if n < 2: return n return fibonacci(n-1) + fibonacci(n-2) print(fibonacci(100)) #
instant -- cached!

```

■ PRACTICE Q & A

Q1. What does 'if `__name__ == __main__`' do?

A: When a Python file is run directly, `__name__` equals '`__main__`'. When it's imported as a module, `__name__` equals the module name. This block contains code that should only run when the file is executed directly, not when imported.

Q2. What is a virtual environment and why do you need one?

A: An isolated Python environment with its own packages independent of the system Python and other projects. Prevents version conflicts between projects. Always use one per project.

Q3. What is the difference between absolute and relative imports?

A: Absolute: `import mypackage.core` (full path from root). Relative: `from . import core` (relative to current file). Relative imports only work inside packages. Absolute imports are generally preferred.

Q4. What does `__all__` do in a module?

*A: Defines the public API -- which names are exported when 'from module import *' is used. If `__all__` is not defined, all names without leading underscores are exported. Best practice: always define `__all__`.*

Q5. What is `@lru_cache` and when should you use it?

A: Memoizes function results -- caches outputs for given inputs so repeated calls with same arguments return instantly without recomputation. Use for pure functions with expensive computation called with repeated arguments (like recursive algorithms).

■ Modules & Packages Cheat Sheet	
<code>import module</code>	Import entire module
<code>from module import name</code>	Import specific name
<code>import numpy as np</code>	Import with alias
<code>if __name__ == '__main__':</code>	Run only when executed directly
<code>__init__.py</code>	Makes directory a package
<code>from . import module</code>	Relative import (inside package)
<code>__all__ = ['fn1', 'fn2']</code>	Define public API
<code>python -m venv venv</code>	Create virtual environment
<code>pip install package</code>	Install dependency
<code>pip freeze > requirements.txt</code>	Save dependencies
<code>@lru_cache(maxsize=None)</code>	Memoize function results
<code>from functools import partial</code>	Pre-fill function args